

Web программирование

Курс для бакалавриата

Чернышев Вячеслав

Тестирование

Содержание лекции

1. Что такое тестирование
2. Unit тесты
3. pytest
4. Генерация тестовых данных
5. E2E с playwright
6. TTD
7. Отчеты о покрытии

Что такое тестирование

Тестирование — это процесс проверки, соответствует ли система или приложение заданным требованиям, и выявления дефектов перед выпуском в эксплуатацию.

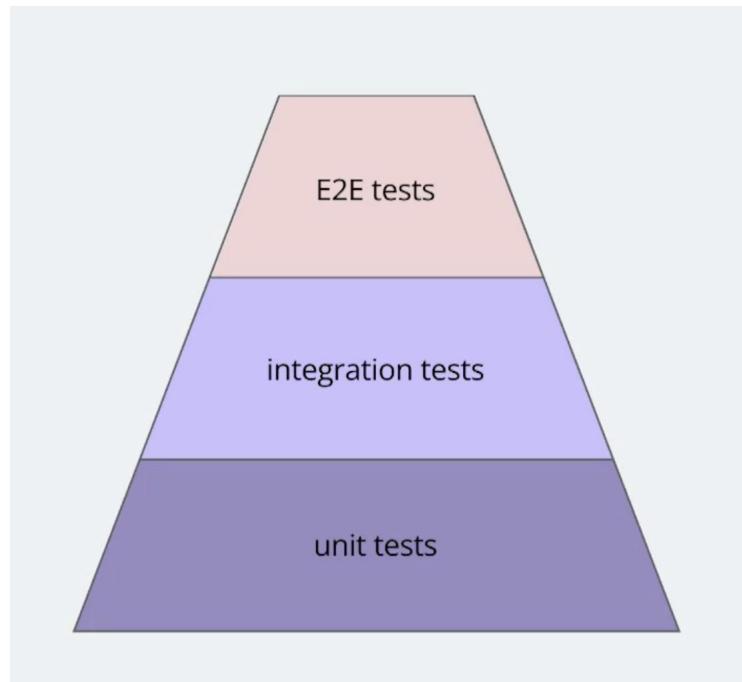
Зачем?

- Обеспечение качества программного продукта
- Обнаружение ошибок и дефектов
- Проверка функциональности, производительности, безопасности и удобства использования

Unit тестирование

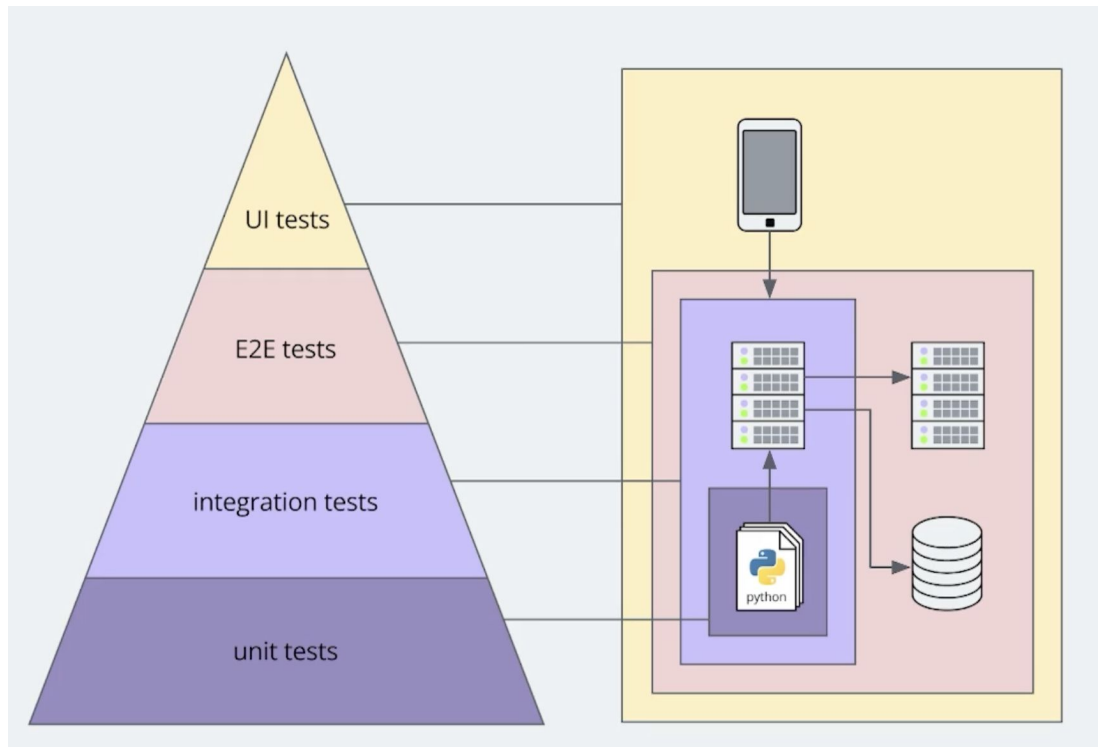
Unit (модульное тестирование)

Способ проверки отдельных модулей системы



Что тестируется

- Отдельные модули/функции/классы
- Только код



pytest

- Простота
- Удобство фикстур
- Параметризованные тесты
- Отдельная обработка stdin/stdout
- Можно запускать мультипроцессорно

```
import pytest

def sum_arif(l: list[int]) -> int: ...

@pytest.fixture
def mylist_10():
    return [i for i in range(10)]

def test_sum_arif(mylist_10):
    print("capture stdout")
    assert sum_arif(mylist_10) == sum(mylist_10)

@pytest.mark.parametrize("l", [
    [i for i in range(N)]
    for N in range(4, 100)
])
def test_sum_arif_parametrized(l):
    assert sum_arif(l) == sum(l)

@pytest.mark.parametrize("l", [
    *[[i for i in range(N)] for N in range(4, 50)]
    + [[10, 0, 5, 2]]
    + [[i for i in range(N)] for N in range(50, 100)]
])
def test_error(l):
    assert sum_arif(l) == sum(l)
```


pytest

pytest -s 01-pytest/[main.py](#)

pytest -v 01-pytest/[main.py](#) - более подробная информация

pytest --maxfail=1 01-pytest/[main.py](#) - прервать если 1 ошибка

```
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson
collected 194 items

01-pytest/main.py capture stdout
.....F.....

===== FAILURES =====
_____ test_error[l46] _____

l = [10, 0, 5, 2]
main.py
@pytest.mark.parametrize("l", [
    *[[i for i in range(N)] for N in range(4, 50)]
    + [[10, 0, 5, 2]]
    + [[i for i in range(N)] for N in range(50, 100)]
])
def test_error(l):
> assert sum_arif(l) == sum(l)
E       assert 24 == 17
E       + where 24 = sum_arif([10, 0, 5, 2])
E       + and   17 = sum([10, 0, 5, 2])

01-pytest/main.py:29: AssertionError
===== short test summary info =====
FAILED 01-pytest/main.py::test_error[l46] - assert 24 == 17
===== 1 failed, 193 passed in 0.06s =====
```

fixtures

Обеспечивают определенный и последовательный контекст для тестов

```
import pytest

class Fruit:
    def __init__(self, name):
        self.name = name

    def __eq__(self, other):
        return self.name == other.name
```

```
@pytest.fixture
def my_fruit():
    print("before")
    yield Fruit("apple")
    print("after")
```

```
@pytest.fixture
def fruit_basket(my_fruit):
    return [Fruit("banana"), my_fruit]
```

```
def test_my_fruit_in_basket(my_fruit, fruit_basket):
    assert my_fruit not in fruit_basket
```

```
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0 -- /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson/venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson
collected 1 item
```

```
02-fixtures/main.py::test_my_fruit_in_basket FAILED [100%]
```

```
===== FAILURES =====
_____ test_my_fruit_in_basket _____
```

```
my_fruit = <main.Fruit object at 0x10aa8ee40>
fruit_basket = [<main.Fruit object at 0x10aad47d0>, <main.Fruit object at 0x10aa8ee40>]
```

```
def test_my_fruit_in_basket(my_fruit, fruit_basket):
> assert my_fruit not in fruit_basket
E assert <main.Fruit object at 0x10aa8ee40> not in [<main.Fruit object at 0x10aad47d0>, <main.Fruit object at 0x10aa8ee40>]
```

```
02-fixtures/main.py:21: AssertionError
```

```
----- Captured stdout setup -----
before
```

```
----- Captured stdout teardown -----
after
```

```
===== short test summary info =====
FAILED 02-fixtures/main.py::test_my_fruit_in_basket - assert <main.Fruit object at 0x10aa8ee40> not in [<main.Fruit object at 0x10aad47d0>, <main.Fruit object at 0x10aa8ee40>]
===== 1 failed in 0.02s =====
```

Время жизни fixture

- **Функции (по умолчанию)** - удаляется после теста
- **Класса** - удаляется после последнего теста класса
- **Модуля** - удаляется после последнего теста в модуле
- **Пакета** - удаляется после последнего теста в пакете
- **Сессии** - живет во время всех тестов

Скоупы (функция vs сессия)

```
import random
import pytest

@pytest.fixture(scope="session")
def sess():
    return random.randint(0, 100)

@pytest.fixture
def func():
    return random.randint(0, 100)

def test1(sess, func):
    raise

def test2(sess, func):
    raise
```

```
sess = 58, func = 49
```

```
def test1(sess, func):
> raise
E RuntimeError: No active exception to reraise

03-fixtures-session-vs-func/main.py:13: RuntimeError
_____ test2
sess = 58, func = 76
def test2(sess, func):
> raise
E RuntimeError: No active exception to reraise
```

Последовательность вызова фикстур

```
import pytest

@pytest.fixture(scope="session")
def order():
    return []

@pytest.fixture
def func(order):
    order.append("func")

@pytest.fixture(scope="class")
def cls(order):
    order.append("class")

@pytest.fixture(scope="module")
def mod(order):
    order.append("module")

@pytest.fixture(scope="package")
def pkg(order):
    order.append("package")

@pytest.fixture(scope="session")
def sess(order):
    order.append("session")

class TestClass:
    def test_order(self, func, cls, mod, pkg, sess, order):
        assert order == ["session", "package", "module", "class", "func"]
```

```
> pytest 04-fixtures-sequence/main.py
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson
collected 1 item

04-fixtures-sequence/main.py . [100%]

===== 1 passed in 0.00s =====
```

Параметризованные тесты

- С помощью fixture(params=
- С помощью mark.parametrize

```
@pytest.fixture(scope="module", params=["smtp.gmail.com", "smtp.yandex.ru"])
def smtp_connection(request):
    smtp_connection = smtplib.SMTP(request.param, 587, timeout=5)
    yield smtp_connection
    print(f"finalizing {smtp_connection}")
    smtp_connection.close()

@pytest.mark.parametrize("conn", ["smtp.gmail.com", "smtp.yandex.ru"])
def test_smtp_connection(conn):
    smtp_connection = None
    try:
        smtp_connection = smtplib.SMTP(conn, 587, timeout=5)
        assert smtp_connection
    finally:
        if not smtp_connection:
            return
        print(f"finalizing {smtp_connection}")
        smtp_connection.close()
```

pytest-asyncio

pip install pytest-asyncio

≡ pytest.ini

```
1  [pytest]
2  asyncio_mode = auto
```

```
import pytest
import asyncio

> async def fetch_data(): ...

> async def add_async(a: int, b: int) -> int: ...

@pytest.mark.asyncio
async def test_fetch_data():
    result = await fetch_data()
    assert result["status"] == "ok"
    assert result["data"] == [1, 2, 3]
```

FastAPI: TestClient

ВАЖНО: нужна библиотека httpx

```
from fastapi import FastAPI
from fastapi.testclient import TestClient
import pytest

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}

@pytest.fixture(scope="session")
def client():
    return TestClient(app)

def test_read_root(client):
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello, World!"}
```


Когда TestClient недостаточно

- Нужно async
- Нужны lifespan события

```
from httpx import AsyncClient, ASGITransport
import pytest

@pytest.fixture(scope="session")
def async_client():
    async with AsyncClient(
        transport=ASGITransport(app)
        base_url="http://testserver"
    ) as client:
        yield client
```

Этапы написания тестов

- Заведение нужных фикстур: инициализация БД, клиент, токены
- Выделение модулей бизнес логики
- Заведение фикстур и тестов для каждого метода

ВАЖНО: желательно все инструменты делать фейковыми

Пример приложения

FastAPI 0.1.0 OAS 3.1

[/openapi.json](#)

default



GET

/public/scrap_users Scrap Users



GET

/admin/users Users



GET

/admin/user/{id} User



PUT

/admin/user/{id} Change User



DELETE

/admin/user/{id} Delete User



Базовый CRUD, который скрапит пользователей с какого-то ресурса

Тестим ручку

```
@pytest.mark.parametrize("limit", [i for i in range(30)])
async def test_scrap(client, limit):
    resp = await client.get("/public/scrap_users", params={"limit": limit})

    assert resp.status_code == 200

    assert len(resp.json()) == limit

    for user in resp.json():
        user_id = user["id"]
        resp = await client.get(f"/admin/user/{user_id}")

        assert resp.status_code == 200
```

Мокаем зависимости

```
@asynccontextmanager
async def lifespan(app):
    settings = Settings()
    if not app.test and not settings.CLIENT_FAKE:
        client = ScrapClient(settings.CLIENT_URL)
        repo = ScrapDataRepository(settings.POSTGRES_DSN)
    else:
        client = FakeScrapClient()
        repo = ScrapDataDictRepository()

    add_dep(ABCScrapClient, client)
    add_dep(ABCScrapDataRepository, repo)
    add_dep(ABCScrapService, ScrapService(repo=repo, client=client))
    yield
```

Определяем тестовую схему БД

```
@pytest.fixture(scope="session")
def db_schema():
    return f"test_{hash(time())}"
```

```
@pytest.fixture(scope="session")
def _settings(db_schema):
    from app.settings import Settings

    s = Settings()
    os.environ["POSTGRES_DSN"] = s.POSTGRES_DSN + f"?search_path={db_schema}"

    return s
```

Определяем тестовую схему БД

```
@pytest.fixture(scope="session")
def db_schema():
    return f"test_{hash(time())}"
```

```
@pytest.fixture(scope="session")
def _settings(db_schema):
    from app.settings import Settings

    s = Settings()
    os.environ["POSTGRES_DSN"] = s.POSTGRES_DSN + f"?search_path={db_schema}"

    return s
```

Создаем/удаляем схему

```
@pytest.fixture(scope="session")
async def raw_pool(db_schema, _settings):
    dsn = _settings.POSTGRES_DSN.replace("+asyncpg", "")
    pool = await asyncpg.create_pool(
        dsn,
        server_settings={"search_path": db_schema},
    )
    print("create schema", db_schema)
    await pool.execute(
        f"""
        create schema "{db_schema}";
        """
    )
    yield pool

    print("drop schema", db_schema)
    await pool.execute(
        f"""
        drop schema "{db_schema}" cascade;
        """
    )
    await pool.close()
```


Создаем таблицы

```
@pytest.fixture(autouse=True, scope="session")
async def tables(_settings, raw_pool):
    from app.db import meta

    event_loop = asyncio.get_event_loop()

    tasks = []

    def run(sql, *multiparams, **params):
        tasks.append(
            event_loop.create_task(
                raw_pool.execute(str(sql.compile(dialect=engine.dialect)))
            )
        )

    engine = create_mock_engine(_settings.POSTGRES_DSN, run)

    print("create tables")
    meta.create_all(engine, checkfirst=False)
    await asyncio.wait(tasks)
    yield

    tasks = []
    print("drop tables")
    meta.drop_all(engine, checkfirst=False)
    await asyncio.wait(tasks)
```

polyfactory

- Генерируем фейковые данные для api
- Вся генерация на основе pydantic моделей

```
from polyfactory.factories.pydantic_factory import ModelFactory
```

```
class FakeScrapClient(ABCScrapClient):  
    class FakeScrappedUser(ModelFactory[ScrappedUser]):  
        @classmethod  
        def email(cls) -> str:  
            return cls.__faker__.email()  
  
    async def user(self) -> ScrappedUser:  
        return self.FakeScrappedUser.build()
```

Polyfactory для редактирования пользователя

```
async def test_change_user_invalid(client, raw_pool, user_factory):
    users = await test_scrap(client, 1, raw_pool)

    user = users[0]
    user_id = user["id"]

    new_user_data = user_factory.build().model_dump()
    new_user_data["id"] = user_id

    resp = await client.put(f"/admin/user/{user_id}", json=new_user_data)

    assert resp.status_code == 422

    assert resp.json() == new_user_data

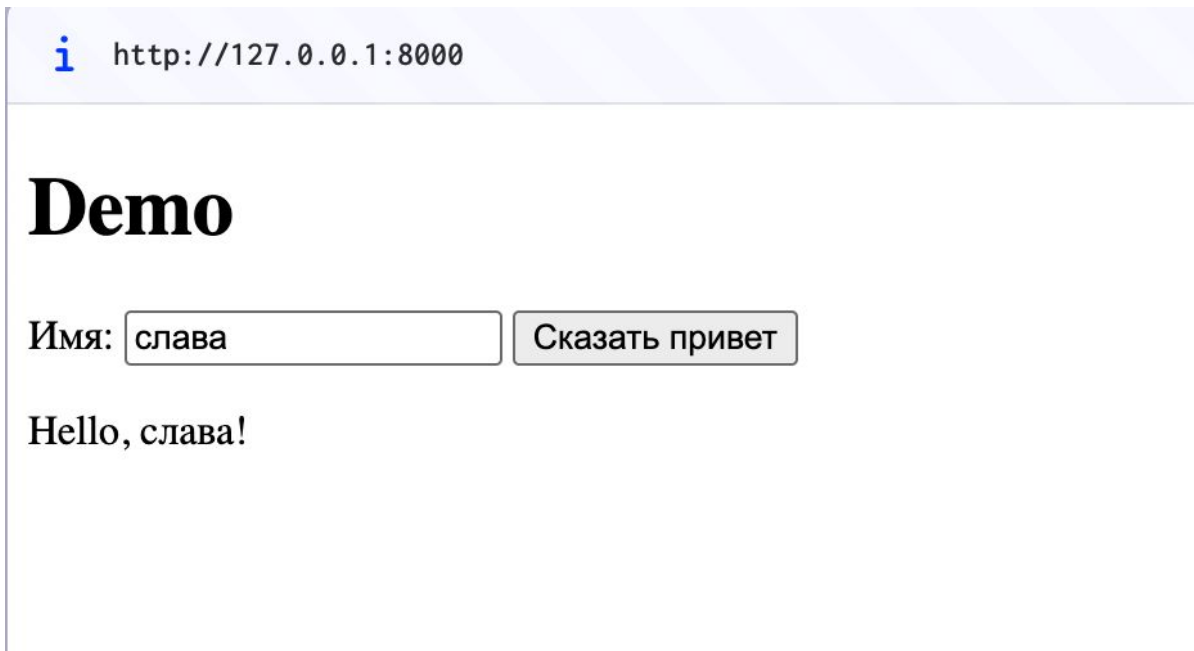
    assert new_user_data != user
```

E2E тесты

- Используется настоящий сервер
- Тестируется реальный пользовательский сценарий
- Выполняются клики, ввод, навигация

Е2Е тесты

- `pip install playwright`
- `python -m playwright install` - ставим браузеры



E2E тесты

Нужен healthcheck

```
def wait_until_server_is_ready(timeout_s: float = 10.0):
    start = time.time()
    while time.time() - start < timeout_s:
        try:
            r = requests.get(f"{BASE_URL}/api/hello?name=test", timeout=0.2)
            if r.status_code == 200:
                return
        except Exception:
            pass
        time.sleep(0.1)
    raise RuntimeError("Server did not start in time")
```

E2E тесты

Запускаем приложение

```
@pytest.fixture(scope="session", autouse=True)
def run_server():
    proc = subprocess.Popen(
        ["python", "-m", "uvicorn", "app:app", "--host", "127.0.0.1", "--port", "8000"],
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
        cwd=APP_DIR,
    )
    try:
        wait_until_server_is_ready()
        yield
    finally:
        proc.terminate()
        proc.wait(timeout=5)
```

E2E тесты

```
@pytest.mark.asyncio
async def test_hello_flow():
    async with async_playwright() as p:
        browser = await p.chromium.launch()
        page = await browser.new_page()

        await page.goto(BASE_URL)

        await page.fill("#name", "Bob")
        await page.click("#btn")

        await page.wait_for_selector("#result")
        await asyncio.sleep(1)

        text = await page.text_content("#result")

        assert text == "Hello, Bob!"
        await browser.close()
```

```
> pytest tests
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson
configfile: pytest.ini
plugins: anyio-4.12.0, asyncio-1.3.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 1 item

tests/test_e2e.py .

===== 1 passed in 1.82s =====
```


TDD

1. Сначала пишем тесты
2. Пишем рабочий код

Зачем:

- Четко понимать требования
- Писать меньше лишнего кода
- Быстро находить ошибки
- Безопасно вносить изменения

Цикл TDD

- Red - пишем тест, он падает
- Green - пишем минимальный код, чтобы тест прошел
- Refactor - улучшаем код, не ломая тесты

TDD - плюсы и минусы

Плюсы:

- Меньше багов
- Код легче поддерживать
- Проще работать в команде

Минусы:

- Выше порог входа
- По началу медленная разработка

TDD

Когда полезен

- Сложная бизнес-логика
- Долгосрочные проекты
- Публичные библиотеки
- Часто меняется код

Когда не использовать:

- Быстрые прототипы
- Экспериментальные фичи
- Лендинги

Информированность о покрытии

pip install pytest-cov

```
> pytest --cov=app
===== test session starts =====
platform darwin -- Python 3.13.1, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/slaveeks/Documents/itmo/webdev-2025/8-lesson
configfile: pytest.ini
plugins: anyio-4.12.0, asyncio-1.3.0, cov-7.0.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=None
collected 7 items

test_app.py ..... [100%]

===== tests coverage =====
_____ coverage: platform darwin, python 3.13.1-final-0 _____

Name      Stmts   Miss  Cover
-----
app.py      20      0   100%
-----
TOTAL       20      0   100%

===== 7 passed in 0.28s =====
```

Информированность о покрытии

`pytest --cov=app --cov-report=term-missing` - показывает непокрытые строки

`pytest --cov=app --cov-report=html` - html отчет

Вопросы?